37. Dedektif Gibi Düşünmek: Debugging'in Psikolojisi ve Bilimsel Metodu

Rastgele Kod Değiştirme Çaresizliğinden Hipotez Testine Geçiş
Bölüm 7 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Junior mühendisler bir bug çıktığında neden rastgele satırları değiştirip dua etmeye başlar? Sherlock Holmes gibi hipotez kurarak hata çözmek nasıl bir his?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Debugging bir şans oyunu değil; bilimsel hipotez kurma ve eleme (Elimination) sürecidir. - 📌 İlk kural: Hatayı deterministik olarak yerel makinenizde yeniden üretebilir (reproduce) hale getirin.
> - 📌 AI modelleri çok iyi bir 'Sanal Dedektif Ortağıdır' (Watson); gözlemlediğiniz belirtileri verip olası şüphelileri listelemesini isteyin.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Sherlock Holmes'un bir cinayet mahalline girdiğini düşünün.

Holmes odaya girip: *"Hadi rastgele sokaktan bir adamı tutuklayalım, belki katil odur. Olmadı başka birini tutuklarız!"* der mi? Tabii ki demez! Çamur izine bakar, kırık vazoyu inceler, saati kontrol eder. İmkansız olan tüm olasılıkları eledikten sonra geriye kalan tek gerçek ne kadar inanılmaz olursa olsun doğru olandır!

Junior yazılımcıların en büyük hatası **"Pompalı Tüfek Debugging" (Shotgun Debugging)** yapmaktır: Oradan bir `+1` sil, şuraya bir `await` ekle, orayı `null` yap… Belki çalışır! Kod daha da batar.

Oysa bilimsel debugging şudur: 1. Gözlem: *"Sadece Salı günleri fatura toplamı 0 çıkıyor."* 2. Hipotez: *"Haftanın günü index'i 0'dan mı başlıyor?"* 3. Test: İzole bir test yaz ve hipotezi doğrula!

🧠 2. AI Çağında Yeni Zihniyet Modeli

Bilimsel Hata Ayıklama Döngüsü:

```
[1. GÖZLEM (Symptom)] ──> "Kullanıcı sepete 2. ürünü ekleyince fiyat ikiye
katlanıyor."
          |
          ▼
[2. TEKRAR ÜRET (Reproduce)] ──> "Adım 1: Giriş yap, Adım 2: X ekle, Adım 3: Y ekle
-> Bug çıktı!"
          |
          ▼
[3. HİPOTEZ KUR (Hypothesis)] ──> "Sepet state'i güncellenirken dizi mutasyona
uğruyor olabilir mi?"
          |
          ▼
[4. TEST ET (Experiment)] ──> "Breakpoint veya izole test ile değişkeni izle."
          |
     ┌────┴─────┐
     ▼          ▼
[DOĞRULANDI]  [ÇÜRÜTÜLDÜ]
(Kök nedeni   (Başka bir hipoteze geç:
 düzelt)       "API yanıtı mı bozuk?")
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Bilimsel Hata Ayıklama ve Dedektiflik Mentoru
Uygulamamızda açıklanamayan bir hata ile karşılaştık.

## BELİRTİLER (Gözlem):
- Kullanıcılar sepete kupon uyguladığında bazen indirim uygulanıyor, bazen de sepet
tutarı 0 TL'ye düşüyor.
- Sorun sadece birden fazla sekme açıkken veya hızlı tıklandığında yaşanıyor.

## GÖREV:
Rastgele kod yazma. Bunun yerine:
1. Bu hatayı açıklayabilecek en mantıklı 3 olası HİPOTEZİ listele (Örn: Race
condition, state mutasyonu, session çakışması).
2. Bu hipotezleri doğrulamak veya çürütmek için hangi adımları izlemem gerektiğini
planla.
3. Hatayı yerel makinemde %100 tekrarlayacak (reproduce) test senaryosunu tarif et.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Pompalı Tüfek Debugging: Sorunu anlamadan rastgele yamalar yapıştırmak!
export function applyDiscountShotgun(cart: any) {
  // "Bazen 0 oluyormuş, o zaman 0 ise eski haline getirelim bari!" (KÖK NEDENİ ÇÖZMEZ!)
  if (cart.total === 0) {
    cart.total = cart.previousTotal || 100;
  }
  return cart;
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
// Kök Neden Analizi Sonucu: Asenkron Race Condition Tespit Edildi ve Çözüldü
export class CartManager {
  private activeDiscountPromise: Promise<number> | null = null;

  async applyDiscountWithLock(cartId: string, couponCode: string): Promise<CartResult> {
    // 1. Kök Neden: Kullanıcı art arda tıkladığında iki istek birbirinin üstüne yazıyordu (Race Condition)
    // Çözüm: Promise Memoization / Mutex Kilidi ile mükerrer istekleri engelle
    if (this.activeDiscountPromise) {
      const pendingTotal = await this.activeDiscountPromise;
      return { cartId, total: pendingTotal };
    }

    try {
      this.activeDiscountPromise = (async () => {
        const cart = await db.cart.findById(cartId);
        const discount = await couponService.calculate(cart, couponCode);
        const newTotal = Math.max(0, cart.subtotal - discount);
        await db.cart.updateTotal(cartId, newTotal);
        return newTotal;
      })();

      const finalTotal = await this.activeDiscountPromise;
      return { cartId, total: finalTotal };
    } finally {
      this.activeDiscountPromise = null; // Kilidi serbest bırak
    }
  }
}
```

💡 **Kod Analizi:** Belirtiyi gizleyen yamalar yapmak yerine kök neden (race condition) bilimsel olarak tespit edilmiş; promise kilitleme mekanizmasıyla sorun kalıcı olarak çözülmüştür.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Hatayı yerel makinede yeniden üretemeden körü körüne koda müdahale etmek.
  - **Çözüm:** Bir hatayı düzeltmeye başlamadan önce onu lokalde %100 tekrarlayan bir test yazın.
- **Tuzak:** Aynı anda 5 farklı dosyayı değiştirip hangisinin işe yaradığını bilememek.
  - **Çözüm:** Her seferinde sadece tek bir değişkeni veya hipotezi değiştirip test edin.
- **Tuzak:** Bug çözülünce regresyon testi yazmadan PR açmak.
  - **Çözüm:** Çözdüğünüz her bug için gelecekte tekrar etmesini engelleyen bir unit test ekleyin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 37: Dedektif Gibi Debugging - Bilimsel Hata Ayıklama*

- 🖥️ **Slayt 1:** Pompalı Tüfek (Shotgun) Debugging neden kariyerinizi bitirir?
- 🖥️ **Slayt 2:** Bilimsel Metot: Gözlem, Hipotez, Deney ve Doğrulama.
- 🖥️ **Slayt 3:** Hatayı %100 yeniden üretilebilir (Reproduce) kılmanın sırrı.
- 🖥️ **Slayt 4:** Canlı Vaka: Açıklanamayan bir sepet hatasını adım adım çözmek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
İçinde gizli bir race condition olan kodu açın. AI'a 'Bu kodda 3 potansiyel hipotez kur' deyin. Hipotezlerden birini test eden küçük bir script yazıp hatayı tetikleyin ve ardından çözümü gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Karşılaştığınız bir sonraki bug'da hemen kod değiştirmeyin. Bir kağıda: 'Gözlemim', 'Hipotezim' ve 'Doğrulama Adımım' başlıklarını yazıp adımları takip edin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Yazılım geliştirmede 'Shotgun Debugging' (Pompalı Tüfek Hata Ayıklama) antipattern'i ne anlama gelir? - A) Bilgisayara ateş etmek - B) Kök nedeni anlamadan ve hipotez kurmadan koddaki satırları rastgele değiştirip şans eseri çalışmasını ummak - C) Sadece Python'da çalışan bir kütüphane - D) Testlerin çok hızlı koşması > **Doğru Yanıt:** B > **Açıklama:** Shotgun debugging, rastgele deneme-yanılma yöntemidir; sorunun kök nedenini çözmediği gibi kodda yeni ve daha gizli bug'lar yaratır.

Soru 2: Bir hatayı çözmeye başlamadan önce yapılması gereken ilk ve en kritik kural nedir? - A) Hemen sunucuyu yeniden başlatmak - B) Hatayı yerel (local) ortamda deterministik olarak %100 yeniden üretilebilir (reproduce) hale getirmek - C) Şirket yöneticisine e-posta atmak - D) Tüm projeyi baştan yazmak > **Doğru Yanıt:** B > **Açıklama:** Tekrarlanamayan (reproduce edilemeyen) bir hatanın çözülüp çözülmediğini bilmek imkansızdır. Önce hatanın oluşma şartları izole edilmelidir.